ChatGPT

Lemon: Network-Wide DDoS Detection via Routing-Oblivious Per-Flow Measurement

Executive Summary: Lemon (USENIX Security 2025) introduces a new data-plane sketch and system for detecting volumetric DDoS attacks **across an entire network** without relying on fixed routing paths [1] [2]. In Lemon, each programmable network device runs a **"Lemon sketch"** that uses *hash-based cardinality estimation* to count packets per flow. Crucially, all devices use the same hash functions so that **updates from the same packet at different points are identical**, enabling a global view without double-counting [3] [4]. A central controller periodically *OR*-merges the sketches from all measurement points and then runs a per-flow query algorithm to estimate each flow's volume (packet count) [5] [6]. Heavy flows (high-volume flows) are also tracked exactly in a small "heavy" table. Lemon was designed for **volumetric multi-path DDoS threats**: attackers may send floods over many routes, so the routing of each packet is **unpredictable** [2]. The goals were (1) **routing-oblivious accuracy** (no assumptions about paths, avoiding over-counting of multi-point observations), (2) **resource frugality** (fits in ~1 MB of fast switch memory), and (3) **scalability** for flexible detection algorithms [4] [7]. We summarize Lemon's architecture, sketch design, and detection logic below, then cover all necessary background (flow definitions, sketching, routing basics, etc.), compare Lemon to related systems (UnivMon, HashPipe, FlowRadar, AM-PM, etc.), and finally discuss practical deployment (parameter choices, evaluation methods, datasets and code).

1. Lemon System Overview

Lemon's **architecture** consists of multiple programmable measurement points (switches or routers) and a centralized controller (Fig. 1). Each device runs the Lemon sketch in its data plane; the controller periodically pulls (or is pushed) the sketches and merges them for network-wide analysis [8]. In this *control-plane* + *data-plane* design, each device independently samples all passing packets into its Lemon sketch, and the controller aggregates them into a single global sketch. Using the globally-merged sketch, the controller runs DDoS detection algorithms (e.g. volume or entropy anomalies) to identify attacks and victims.

- **Design Goals:** As the paper states, Lemon is targeted at **volumetric DDoS attacks** where the goal is to flood a victim's bandwidth [2]. In particular, Lemon assumes the attack traffic may traverse *multiple* routes and measurement points, with *unknown and changing* paths [2]. To succeed, Lemon must avoid **over-counting** (counting the same packet twice if it is seen at multiple points) and **mis-allocating** (incorrectly merging per-switch counts for the same flow) [4] [9]. The design goals are thus:
- **Routing-Oblivious Measurement:** Accurate per-flow counts without any knowledge of routing. This requires data structures and aggregation algorithms that inherently handle duplicate observations [4] [9].
- **Resource-Efficiency:** All per-packet work must fit within typical switch constraints (e.g. ~1 MB on-chip memory, line-rate operations) [4] [9].
- **Flexibility and Scalability:** The system should support arbitrary flow identifiers (e.g. IPv4 five-tuple or other key fields) and allow various DDoS detection methods. The control plane must flexibly configure which flows to monitor and how to detect attacks [10].

- **Threat Model:** Attackers generate massive packet floods to target IP(s) (potentially many victims) via multiple network paths. The defender controls the switches, which are not compromised. The measurement points see all traffic (each packet should hit *at least one* measurement point) [7]. We assume flows and packets can be identified (e.g. using a flow key like 〈srcIP,dstIP,srcPort,dstPort,protocol〉 and a unique packet ID such as a checksum) [11]. Attack traffic can spoof source IPs, use many sources, or split across ECMP. Lemon makes *no assumptions* about routing: any packet may be seen at one or more points.

- **Architecture Details:** Figure 1 illustrates the high-level view. Each switch's **data plane** contains one or more registers (arrays of bits) for the Lemon sketch. It also maintains a small *heavy-flow table* mapping any flow identified as "heavy" to an exact packet count (the heavy table is empty by default). No other per-flow state is kept. The controller (CP) periodically collects the raw sketches (bitmaps) and heavy tables from all switches. It then performs a **bitwise OR** across the sketches to form a single *global sketch* (bit OR is safe because each switch uses identical hash updates for the same packet). The heavy tables are merged by summing counts per flow. Finally, the CP can run arbitrary detection logic (e.g. find flows whose estimated volume exceeds a threshold, or compute entropy across destinations).

【Figure 1: Lemon's architecture. Network traffic flows through multiple switches (each running the Lemon sketch). A centralized controller regularly fetches sketches from all switches, ORs them into a global sketch, and then queries it to estimate each flow's volume for DDoS detection.】

（The diagram is conceptual; actual deployment might have dozens of switches.）

Key Assumptions and Limitations: Lemon assumes all measurement switches are programmable (e.g. P4-capable) and cover the network so that each packet is seen at least once [7]. It assumes flows can be uniquely identified and that packets within a flow are distinguishable (e.g. by a sequence number or checksum). Lemon focuses on volume-based DDoS; very stealthy attacks (with very low per-packet payload or timing-based attacks) are out of scope. Because Lemon uses probabilistic sketches, there is a small error in estimates (especially for small flows or when memory is very limited). In practice, the paper shows collision rates <1% with sane parameters, and heavy flows are counted exactly [12] [13]. Finally, Lemon (as presented) was prototyped on synthetic and MAWI traces; real large-scale deployment may require tuning (but no fundamental obstacle is known).

2. Routing-Oblivious Per-Flow Measurement in Lemon

The heart of Lemon is the **Lemon sketch**, a novel multi-layer bit-sketch that **estimates each flow's packet count** in a distributed, routing-oblivious manner. Here we describe its algorithms, data structures, and operation in detail.

2.1 Data Structures and Update Rules

The Lemon sketch is composed of *n* layers (e.g. 5–6 layers). Each layer *i* has a fixed number of "units" (slots) and each unit is a small bit-array of *S'_i* bits. We also choose threshold values *T₀=H_d.max, T₁, …, T_n=0* that partition the range of a 32-bit hash (or similar). For example, one configuration might use 5 layers with T-values [16,384; 4096; 1024; 256; 0] and slot counts [524,288; 65,536; 8,192; 2,048; 1,024] [14].

Hash Functions: Lemon uses **three** independent hash functions on each packet *p*:
- *H_d(pktID)*: a hash of the packet's unique identifier (e.g. sequence number), producing a

large integer in [0, H_d.max). We call this *h_{layer}* = H_d(pktID).
- *H_s(flowID)*: a hash of the flow identifier (e.g. the five-tuple), yielding *h_{slot}*.
- *H_b(pktID)*: another hash of the packet ID, yielding *h_{bit}* for selecting a bit.

All switches use the **same** hash functions and parameters. This ensures consistent updates: the same packet at two switches will generate identical *h_{layer}*, *h_{slot}*, and *h_{bit}*.

Updating the Sketch (Per-Packet): When a packet with ⟨flowID, pktID⟩ arrives at a switch, the Lemon sketch is updated as follows (see Algorithm 1). First, compute *h_{slot} = H_s(flowID)* and *h_{layer} = H_d(pktID)* and *h_{bit} = H_b(pktID)*. Now find which layer *i* this packet maps to: find the unique *i* such that

T_i ≤ h_{layer} < T_{i−1}.

If none (i.e. if *h_{layer}* ≥ *T₀* or *h_{layer}* < *T_n*), we treat the packet as "overflow" (but in practice *T₀=H_d.max*, *T_n=0* so every value fits exactly one layer). Suppose the chosen layer is *i*. Then we compute a slot index *idx = h_{slot} mod S_i* and a bit index *j = h_{bit} mod S'_i*. We then **set** bit *j* in unit *idx* of layer *i* to 1. (This is done by a simple register write in the switch.) Importantly, for each packet **only one** layer gets updated (the one matching *h_{layer}*). This is effectively a form of *hash-based sampling*: different layers correspond to different sampling rates.

Heavy-Flow Table: Additionally, Lemon maintains a small exact-count table for heavy flows. If *h_{layer}* falls into the bottom-most layer (i.e. *h_{layer}* < *T_n*), we conclude the packet belongs to a very large flow (beyond all thresholds), and we increment a precise counter for that flow in the heavy table [15]. Thus very heavy flows are handled exactly rather than probabilistically.

Algorithm 1 (Lemon sketch update) is as follows (pseudocode):

```
Input: packet with (flowID, pktID)
Output: updated Lemon sketch and heavy table

h_slot  = H_s(flowID)
h_layer = H_d(pktID)
h_bit   = H_b(pktID)
for i = 1 to n_layers:
    if (T_i ≤ h_layer < T_{i-1}):
        slot = h_slot mod S_i
        bit  = h_bit mod S'_i
        bitmap[i][slot][bit] = 1   // set bit
        break
if (h_layer < T_n):              // if reached below last threshold
    heavy_table[flowID] += 1     // count packet in heavy flow table
```

Because all switches use the **same** H_s, H_d, H_b, the effect of this update at different switches for the same packet is identical. For example, if a packet for flow X is seen at two routers, both routers will set the **same** bit in the same layer. Thus, when we later OR their

sketches, that packet contributes only once. This is the key to **over-counting-free** (routing-oblivious) measurement [3] [4].

2.2 Querying the Sketch (Per-Flow Count Estimation)

At the controller, we merge sketches by bitwise OR (and sum heavy counters). To estimate the volume (packet count) of a specific flow, Lemon runs a **query algorithm** on the merged sketch (Algorithm 2). Given a flowID, we recompute *h_{slot} = H_s(flowID)* (note: packet hashes H_d and H_b are not needed since we already encoded their effect). Then for each layer *i*, we extract the bit-array of layer *i* at index *(slot = h_slot mod S_i)*; call this *bitmap_i*. We count the number of 0-bits in that array, *Z_i*.

Intuitively, if flow *f* has *N* packets, roughly *N / (T_{i-1} − T_i)* of them were hashed into layer *i*. A bitmap of size *S′_i* with *N_i* random bits set will have $Z_i \approx S'_i \cdot \exp(-N_i/S'_i)$ zeros (by the "linear counting" formula [16]). Lemon takes the **first** (lowest) layer *i* for which the fraction of zeros Z_i/S'_i is **at least** a threshold T_z (e.g. 0.2 to avoid near-saturation) [6]. We then estimate $N_i \approx -S'_i \cdot \ln(Z_i/S'_i)$, and scale it up by the inverse sampling probability to get the final estimate *E*. Concretely:

1. For *i=1..n*, compute *Z_i = count_zeros(bitmap_i)* [6].
2. Find the first layer *i* such that $Z_i \geq T_z \cdot S'_i$ (i.e. not too many bits set).
3. Compute $E = -S'_i \cdot \ln(Z_i / S'_i) \cdot (H_d.max/(T-T_i))$ [16]. (Here *H_d.max/(T_{i-1}-T_i)* adjusts for the fraction of packets hashed to this layer.)
4. Output *E* (and record that layer *i* was used).

If *no* layer satisfies $Z_i \geq T_z S'_i$, we declare an overflow (the flow is extremely large, which in practice will have been caught by the heavy table anyway). Note that the **heavy-table** exact count is used if flowID was ever flagged as heavy; otherwise we rely on *E* from the sketch.

Algorithm 2 (Lemon sketch query) summary:

```
Input: flowID
Output: estimated packet count E (or “overflow”)

h_slot = H_s(flowID)
for i = 1 to n_layers:
    slot = h_slot mod S_i
    bitmap_i = layer_i[slot]        // array of S'_i bits
    Z_i = count_zeros(bitmap_i)
    if (Z_i >= T_z * S'_i):
        E = - S'_i * ln(Z_i / S'_i)
        E = E * (H_d.max / (T_{i-1} - T_i))
        return E
return overflow
```

This method effectively implements a form of **sampled distinct counting**: different layers correspond to different sampling rates, and we pick the layer that just avoids saturation. The multiplicative factor *(H_d.max/(T_{i-1}-T_i))* corrects for the fact that only a fraction of packets land in layer *i*.

2.3 Error Bounds, Memory/CPU Costs, and Collision Handling

Accuracy: In practice, Lemon can achieve high accuracy. The linear-count estimator $-S' \cdot ln(Z/S')$ is unbiased with known variance. By choosing thresholds T_i and sizes S_i, S'_i appropriately, the paper shows that Lemon's **precision and recall** for detecting heavy flows or "superspreaders" is near 100% for moderate sketch sizes [17] [18]. For example, on real MAWI traces (≈200k distinct dest IPs) with ~1 MB of memory, Lemon's weighted absolute percent error (WAPE) on per-flow volume stayed very low; only when memory dropped below ≈0.4 MB did collision errors rise significantly [18]. (Above 0.4 MB, almost no flows collided in a way that harmed the estimates.) Theoretical analysis in the paper shows collision rates of well under 1% for each layer with typical workloads [19].

- **Collision Elimination:** Even if two flows hash to the same slot in a given layer, Lemon handles it gracefully. The key is that flows of **different sizes** tend to "settle" in different layers. If a small flow and a large flow collide in a low layer unit, the algorithm will likely find that the large flow is estimated at a higher layer. In effect, the smaller flow's estimate is not affected by the large flow's extra bits, and vice versa [20] [13]. Section 4.5 of the paper proves that when two flows have different best-sample layers, their estimates do not interfere. In practice, empirical collision rates are very low (<1% per layer [19]) under realistic traffic patterns, and the post-processing corrects most residual errors.

- **Memory & Computation:** Lemon is extremely lightweight. The data plane sketch requires only bit-arrays (~1 MB total in the example configuration) plus a tiny heavy-flow table (only for the rare large flows). The authors report an example: five layers with S′ = [16K,4K,1K,256,?] and S = [524K,65K,8K,2K,1K] (bits and slots) uses on the order of 8 million bits (≈1 MB) [19]. In hardware (Intel Tofino, as implemented), Lemon uses only 2–3 pipeline stages and a handful of simple hash operations per packet [21]. Each packet update does one hash for slot and layer and one register write (constant time). The control-plane merging is a bitwise OR of sketches (fast with bitwise ops) and summing of a few counters.

- **Error Bounds:** The estimation error from the linear-count formula is on the order of $O(1/S')$ for large flows. In practice, the paper shows per-flow relative error (RE) often below a few percent, and detection F1-scores near 1.0 [18]. The detailed analysis (Theorem 1 in the paper) gives the probability distribution of estimating a flow at a given layer and the collision probabilities; under typical settings these are negligible. In sum, Lemon achieves **high accuracy** for large and moderate flows, and is conservative (often slightly undercounting) due to hashing.

- **Summary of Lemon Sketch:** Lemon sketch is essentially a **routing-oblivious distributed counting sketch**. It sacrifices a bit of precision for tiny memory, avoids any per-flow state in the data plane, and neatly handles multi-path updates. By using consistent hashing and bitmaps, it guarantees that each packet contributes exactly as once network-wide. The control plane then can **query any flow key** for its estimated volume. This enables per-flow DDoS detection: e.g. the controller can treat each destination IP as a "flow" key and find which dest IPs have unusually high inbound packet counts.

3. Background and Prerequisites (Teaching Materials)

To fully understand Lemon, we review key concepts in network flows, measurement, sketches, and routing.

- **IP Flow Definition:** In networking, a *flow* is typically defined by a set of packet header fields. The classic definition is the 5-tuple ⟨source IP, dest IP, source port, dest port, protocol⟩, which identifies a unidirectional TCP/UDP conversation. Lemon (and many DDoS systems) often use simpler keys: for volumetric DDoS, one may treat all packets with the same *destination IP* as a "flow" (i.e. flooding to a victim). One can also define flows by ⟨srcIP,dstIP⟩ or just ⟨dstIP⟩. The flow identifier is an input to a hash.

- **Flow Measurement (NetFlow/sFlow):** Traditional network monitoring (NetFlow, IPFIX) collects per-flow counters by logging each flow's start, end, and byte count at routers. This requires per-flow state and can overwhelm CPUs/DRAM in high-speed networks. NetFlow is often implemented by sampling or timeouts. An alternative is *sFlow*, where routers randomly sample 1-in-N packets and send them to a collector; the collector extrapolates flow rates. Both approaches can fail under high load or fine timescales, and they are not inherently routing-oblivious.

- **Sketches and Counters:** To reduce memory, **sketches** (compact data structures) are used. For example, a *Count-Min sketch* uses a 2D array of counters with multiple hash functions to estimate flow frequencies (with over-estimation bias). A *Bloom filter* uses a bit-array to test membership (with false positives). Lemon's sketch is related to *linear counting* and *HyperLogLog*-style counting: a bit-array per flow to estimate distinct count of packet IDs.

- **Heavy-Hitter Detection:** A **heavy-hitter** is a flow whose traffic volume exceeds a threshold (or rank). Algorithms like Space-Saving or Counter *tables can detect heavy hitters in a single stream. Lemon aims to detect network-wide heavy hitters** by combining measurements from all switches.

- **Superspreaders:** In DDoS, one also cares about *superspreaders* (destinations receiving packets from many distinct sources) or many flows. This is similar to counting distinct elements (like HyperLogLog). Lemon's bitmaps can also be used for distinct-source counts if configured appropriately (e.g. using source IP as packet key).

- **Distributed Aggregation:** When multiple switches each measure traffic, naive aggregation (just summing counts) double-counts packets that traverse multiple switches. A *routing-aware* scheme might track which flow portion went where, but Lemon's whole point is to be **routing-oblivious**: no per-path information is needed. Lemon achieves this by making each packet's contribution identical at all points (so OR-sum works). Alternative approaches include: *packet marking* (tagging packets with bits at one point so others skip them) [22], or *coordination* between switches (like cSAMP's responsible partitioning) [23]. Lemon's approach fits well with modern P4 switches: no packet header modification, just in-switch state.

- **Routing Basics:** In IP networks, routers use protocols (OSPF, BGP, ECMP) to forward packets. If multiple equal-cost paths exist (ECMP), flows are typically hashed onto one path. Importantly, *different routers may see different subsets of a flow's packets*. Even a single TCP flow might alternate paths if the network rebalances. Thus any measurement that simply sums each router's count can over-count. Lemon's design makes *no assumption about which path packets take*: it treats the network as one logical switch where each packet is processed identically.

- **BGP/ECMP:** For completeness: BGP handles inter-domain routing, not directly used here. The key idea is that routing can change dynamically (load balancing, link failures). Lemon assumes the attacker may exploit this: flood traffic could be split. Thus **static routing assumptions do not hold**.

- **DDoS Detection Metrics:** To evaluate detection, common metrics are **precision, recall, F1** (harmonic mean). If we label flows "attacking" vs "benign" (e.g. above/below a volume threshold), Lemon's output can be scored by these metrics. For volume estimation, one uses **relative error (RE)** between estimated and true counts.

- **Sketch Examples:** To illustrate, consider a simple 1-layer Lemon (like a Bloom filter for one flow). Suppose $S'=100$ bits, $S=10$ slots. If a flow has 20 packets, we might set ~12 bits ($Z\approx 88$). The estimate $E=-100\cdot\ln(88/100)\approx 12$ (needs scaling). In reality, Lemon uses multiple layers to accommodate flows from 1 packet up to millions, automatically choosing the right "sampling rate" per flow.

- **FlowRadar:** For background, FlowRadar (NSDI 2016) is a sketch that encodes all active flows and counts in a switch using an *invertible Bloom lookup table*. It can exactly recover flows under low load, but it assumes each flow stays on one switch (it is not routing-oblivious). It also uses ~2–3 MB per switch for 100K flows [24].

- **HashPipe:** Another concept: HashPipe (SIGCOMM 2017) uses per-switch registers to find the top-k heavy flows *locally*. It can detect ~95% of the largest flows with only ~80 KB [25]. However, HashPipe reports local heavy flows and does not by itself correlate them network-wide.

- **Alternate Marking (AM-PM):** AM-PM is a different telemetry technique (IETF RFC 9341) for measuring *performance (loss/delay)* by toggling one header bit in packets at fixed intervals [26]. It's *routing-oblivious* (bit is cleared/set regardless of path), but it does **not measure per-flow volumes** – it's for path loss. We list it mainly as a contrast in techniques.

In summary, Lemon builds on a rich ecosystem of sketches and telemetry methods, but its novelty is the combination: **network-wide, per-flow measurement that is robust to any routing**.

4. Comparison with Related Work

We compare Lemon to several key alternatives for large-scale flow measurement and DDoS detection. Table 1 summarizes attributes. In brief:

- **UnivMon [2] (SIGCOMM 2016)** – a "universal" streaming sketch that can estimate many per-flow quantities. It requires static routing assumptions (each flow seen once) and has high memory usage for many flows, though its accuracy is good. UnivMon is not routing-oblivious and is complex to deploy network-wide [27].
- **HashPipe [19] (SIGCOMM 2017)** – extremely lightweight heavy-hitter detector. Per-switch, memory ~80 KB finds ~95% of top-300 flows [25]. It is *not* network-wide: cannot correlate flows across switches, and it is not routing-oblivious (simple summation would double-count).
- **FlowRadar [23] (NSDI 2016)** – an invertible sketch for per-flow counters. It can (in one switch) recover counts for up to ~100K flows using ~2.3–2.9 MB [24]. Accuracy is perfect for decoded flows, but it **assumes flows stay on one device** (not oblivious) [28]. If flows split or change paths, FlowRadar cannot merge them correctly.

- **AROMA [35] (INFOCOM 2020)** – a routing-oblivious flow sampling system by Ben-Basat et al. It hashes packets uniformly across switches and then re-assembles samples at the controller, guaranteeing uniform sampling without coordination. AROMA works in arbitrary topologies, but its raw scheme requires tens of MB on-chip (e.g. 16 MB IPv4) [29] to function and only supports sampling-based estimates.
- **SAROS [40] (APNet 2024)** – a recent routing-oblivious sampler by Wu et al. (same group as Lemon). SAROS also exploits sampling in the data plane, with adaptive thresholds set by the controller. It uses very little memory on the switch (< few KB per layer) by only keeping counters of sampled packets. SAROS improves heavy-hitter F1 (by 10–40%) over prior methods by addressing double-counting, but it *samples* packets and thus requires careful parameter tuning. It still leaves some recall loss for medium flows.
- **Network-Wide HH (CDM) [27] (SOSR 2018)** – an SDN-based scheme by Rexford et al. that treats the network as one big switch and uses **adaptive thresholds** at each switch to catch large flows. It is *routing-aware* (it knows network geometry) and sends exact counts to the controller. It achieves savings vs. naive reporting, but requires coordination (and a controller). It is not oblivious to dynamic routing: if flows migrate, the scheme can double-count unless thresholds adapt in real-time.
- **AM-PM (RFC 9341)** – not a heavy-hitter method, included for contrast. Uses 1-bit marking to measure path loss/delay. Routing-oblivious for its purpose, but it does *not* give flow volumes or detect DDoS by volume. It has negligible per-packet cost but cannot identify heavy flows.

Table 1: Comparison of Lemon vs. alternative techniques. (Attributes: granularity refers to whether per-flow counts are available, routing-obliviousness means correct under arbitrary routing, memory = typical per-device on-chip usage, accuracy = detection/estimation quality, complexity = deployment difficulty, scalability = ability to handle many flows/switches. The entries are qualitative, e.g. "Low/Med/High" or "Yes/No".)

System	Detection Granularity	Routing-Oblivious?	Memory (per device)	Accuracy (DDoS detect)	Deployment Complexity	Scalability
Lemon (ours)	Per-flow (any 5-tuple)	**Yes**	Low (≈1 MB)	Very high (near 100% F1)	Moderate (P4+ controller)	High (many switches, flows)
UnivMon	Per-flow sketch	No (static routing)	High	High (sketch-based errors)	High (complex sketch)	High (theoretical)
HashPipe	Local heavy-hitter	No	Very low (~80 KB)	High for top flows	Low (simple P4 tables)	Medium (only local)
FlowRadar	Per-flow counters	No (single switch)	Medium (≈2–3 MB)	High if decodable	Medium (collector needed)	High (per-switch, flows)
AROMA	Flow sampling	**Yes**	Very high (tens MB)	High (sampling error)	High (multi-stage P4)	High (any topology)

System	Detection Granularity	Routing-Oblivious?	Memory (per device)	Accuracy (DDoS detect)	Deployment Complexity	Scalability
SAROS	Heavy-hitter (samples)	**Yes**	Low (few KB)	Medium-high (improved HH)	Medium (dynamic config)	Medium (some coordination)
NW Heavy-Hitter (CDM)	Per-flow if above threshold	No	Low (counters/ thresholds)	Medium (struggles if routing changes)	Medium (SDN controller)	Medium (depends on deployment)
AM-PM	Path performance (loss/delay)	Yes	Negligible (bit marking)	N/A (not for volume)	Low (1-bit config)	High (standardized)

Table 1: Comparison of Lemon and related approaches. Lemon is unique in offering true *network-wide per-flow counts* with routing-oblivious guarantees, at very low memory cost [4] [7]. In contrast, UnivMon-like sketches and FlowRadar assume stable single-switch paths [28], while HashPipe only finds local top flows [25]. AM-PM is unrelated to heavy hitters. AROMA/SAROS are recent routing-oblivious samplers; Lemon's advantage is exact per-flow counting (within sketch error) rather than sampling, at modest overhead.

5. Practical Guidance for Deployment and Evaluation

Parameter Selection: In deploying Lemon, one must choose the number of layers, layer thresholds T_i, number of slots S_i, and bit-array sizes S'_i. The goal is to cover the expected flow size range. For example, the paper's default had 5 layers with $T = [16384, 4096, 1024, 256, 0]$ and slots $S = [524288, 65536, 8192, 2048, 1024]$ [19]. Smaller flows (≤10–100 packets) will end up in the lowest layer(s), larger flows in higher layers. One should ensure S'_i is large enough (bits) to avoid saturating: the paper uses a threshold $T_z = 0.2$ to skip layers with >80% bits set. A rule of thumb: give lower layers many slots but few bits (since many small flows), and higher layers fewer slots but more bits (since few large flows).

The heavy-flow threshold (implicit T_n) should be set at the size above which exact counting is affordable. In practice, flows that fill the last bitmap layer are counted exactly. Switch memory limits (~1–2 MB on-chip) typically constrain total $\sum S_i \cdot S'_i$. We recommend starting with the example configuration and adjusting based on traffic patterns. If traffic has more tiny flows, allocate more slots in layer 1; if many mid-sized flows, add a layer. The hash functions must be uniform and consistent (e.g. CRC-based).

Deployment: Lemon was implemented in P4 for BMv2 (software switch) and for an Intel Tofino ASIC [30]. The code is ~500–600 lines of P4 per target, plus controller scripts [30]. One can deploy Lemon on any P4-capable switch or simulate on BMv2. The controller must collect sketches from all switches; this can be done via periodic RPC or by embedding in telemetry (e.g. INT). The GitHub repository provides working examples: under `lemon_bmv2` and `lemon_hardware` folders [31]. The controller code includes an attack traffic generator and analysis scripts.

Data Generation and Datasets: To test Lemon, one needs a mix of benign and attack traffic:
- *Benign traffic:* Real-world traces (e.g. CAIDA or MAWI archives) provide realistic background. The Lemon

authors used the MAWI Working Group trace [32]. For example, 5 million packets from a WIDE/MAWI trace (~7 Gbps over 5 s) was used as background [33]. CAIDA UCSD traces (DDoS victim traces) can also be used.
- *Attack traffic:* One can generate synthetic UDP floods or distributed floods. The Lemon prototype included a generator script (see `/controlplane/traffic_generation/`) [32]. Vary the attack intensity from a fraction of total traffic up to tens of percent. To simulate spoofing, randomize source IPs or use a botnet model. Lemon evaluation varied attack volume from 0.1% to 10% of total [34].

Evaluation Metrics: Key metrics are detection **F1-score** (for correctly finding attack flows or victims) and **relative error** (RE) in estimating volumes. For heavy-hitter tasks, one measures recall (fraction of true heavy flows found) and precision (false positives), then F1 [35]. The relative error RE = |estimate–true|/true is used for volume estimation [36]. Plotting F1 vs. switch memory or vs. number of measurement points is common.

Recommended Experiments:
1. **Accuracy vs Memory:** Fix the network and traffic; vary the total sketch memory (by adjusting layer sizes). Plot estimation RE or detection F1 as memory increases. We expect recall/f1 to rise steeply and then plateau once memory is sufficient [18]. Compare Lemon vs. a strawman (e.g. Count-Min or UnivMon).
2. **Routing Dynamics:** Emulate route changes or ECMP splits in the network. Send traffic over multiple paths. Verify that Lemon's counts remain accurate (consistent) while a naive aggregator would double-count. For example, send the same heavy flow through two switches and check Lemon's OR-merge yields the correct total.
3. **Topology Variance:** Test on different topologies (e.g. tree, mesh, ISP). Ensure that as long as switches see traffic, Lemon works. The evaluation in the paper used 8 and 100-node networks with random traffic [19].
4. **Baseline Comparisons:** Implement a naive solution (e.g. each switch counts flows and controller sums them). Show how that misses attacks (due to under-counting at any single switch) or double-counts under multi-path. Compare to Lemon's output.
5. **Parameter Sweeps:** Evaluate different numbers of layers, different *T_i* thresholds, and different *T_z*. See how these affect collision rates and error.
6. **Real vs Synthetic:** Use real traffic from MAWI or UNIBS; overlay synthetic DDoS. Also test purely synthetic Zipf workloads (the paper used Zipf flows to stress small/large flows) [37].
7. **Scalability:** Increase number of measurement points. Lemon should still work as long as sketches are merged. Test with partial deployment: only a subset of nodes running Lemon and see how that reduces accuracy.

Code and Datasets: The Lemon artifact (code and scripts) is publicly available [38]. The code repository's README links to the MAWI trace used [32]. Open datasets: WIDE/MAWI traces are free to download [39]. The CAIDA DDoS traces (such as the 2016 DDoS traces) are also commonly used. For synthetic traffic, tools like **tcpreplay** or **Scapy** can generate flows.

【Figure 2: Lemon data flow on one switch. A packet's flowID and packetID are hashed into a layer, slot, and bit. The corresponding bit in the sketch is set, and possibly a heavy-counter is incremented.】

```
flowchart LR
  Packet["Packet<br/><code>(flowID, pktID)</code>"] -->|H<sub>s</
sub>(flowID)| HashSlot["Hash to slot index"]
  Packet -->|H<sub>d</sub>(pktID)| HashLayer["Hash to layer selector"]
  Packet -->|H<sub>b</sub>(pktID)| HashBit["Hash to bit index"]
```

```
HashSlot --> SelectSlot["Compute slot = h_slot mod S_i"]
HashBit --> SelectBit["Compute bit  = h_bit mod S'_i"]
HashLayer --> SelectLayer["Choose layer i s.t. T_i ≤ h_layer < T_{i-1}"]
SelectLayer --> UpdateBit["Set bit in that layer and slot"]
UpdateBit --> CheckHeavy{"If h_layer < T_n?"}
CheckHeavy -- Yes --> IncHeavy["Increment heavy_table[flowID]"]
CheckHeavy -- No --> End
IncHeavy --> End[End]
```

6. Sources and Further Resources

- **Lemon (Wu et al., USENIX Security 2025)** – Original paper and source code [1] [38]. Key for architecture, sketch design, evaluation.
- **GitHub Lemon Repository** – Implementation and artifacts [38]. Includes P4 code and dataset scripts (MAWI reference) [32].
- **MAWI/WIDE Traffic Archive** – Background traffic dataset used in evaluation [32] [39].
- **Jaqen (Liu et al., USENIX SEC 2021)** – Switch-native DDoS detector using UnivMon (for comparison of switch-based methods) [40].
- **FlowRadar (Li et al., NSDI 2016)** – Invertible sketch for per-flow counters [41] [24].
- **HashPipe (Moshref et al., SOSR 2018)** – Heavy-hitter detection in P4 [25].
- **UnivMon (Liu et al., SIGCOMM 2016)** – Generic sketch for multiple metrics.
- **AROMA (Ben-Basat et al., INFOCOM 2020)** – Routing-oblivious sampling sketch [42] [43].
- **SAROS (Li et al., APNet 2024)** – Self-adaptive sampling heavy-hitter detector [44].
- **Network-wide Heavy-Hitter (Harrison et al., SOSR 2018)** – Threshold-based HW heavy-hitter detection [45].
- **Alternate Marking (RFC 9341, IETF 2022)** – 1-bit loss/delay telemetry [26].
- **Data Streaming Texts:** Cormode & Muthukrishnan, “Data Stream Summaries” (heavy hitters/sketches); Mitzenmacher & Upfal, “Probability and Computing” (hashing/flajolet-martin).

Prioritized References: Wu *et al.* 2025 (Lemon, primary) [1] [4]; GitHub f-555/Lemon [38] [32]; Li *et al.* 2016 (FlowRadar) [24]; Moshref *et al.* 2018 (HashPipe) [25]; Harrison *et al.* 2018 (Net-wide HH) [45]; Ben-Basat *et al.* 2020 (AROMA) [42]; Li *et al.* 2024 (SAROS) [44]; IETF RFC 9341 (AM-PM) [26]; plus the MAWI trace site [39].

---

[1] Lemon: Network-Wide DDoS Detection with Routing-Oblivious Per-Flow Measurement | USENIX
https://www.usenix.org/conference/usenixsecurity25/presentation/wu-wenhao

[2] [3] [4] [5] [6] [7] [8] [9] [10] [11] [12] [13] [14] [15] [16] [18] [19] [20] [27] [30] [33] [34] [36] [37] usenix.org
https://www.usenix.org/system/files/usenixsecurity25-wu-wenhao.pdf

[17] [21] [22] [23] [28] [35] [42] [43] Routing_Oblivious__SOSR_2020_(2).pdf
https://minlanyu.seas.harvard.edu/writeup/ifip20.pdf

[24] [41] usenix.org
https://www.usenix.org/system/files/conference/nsdi16/nsdi16-paper-li-yuliang.pdf

[25] [1611.04825] Heavy-Hitter Detection Entirely in the Data Plane
https://arxiv.org/abs/1611.04825

[26] RFC 9341: Alternate-Marking Method
https://www.rfc-editor.org/rfc/rfc9341.html

29 44 SAROS: A Self-Adaptive Routing Oblivious Sampling Method for Network-wide Heavy Hitter Detection
https://conferences.sigcomm.org/events/apnet2024/papers/SAROSASelf-AdaptiveRoutingObliviousSamplingMethodforNetwork-wideHeavyHitterDetection.pdf

31 32 38 GitHub - f-555/Lemon: Network-wide DDoS Detection with Routing-Oblivious Per-flow Measurement · GitHub
https://github.com/f-555/Lemon

39 MAWI Working Group Traffic Archive
https://mawi.wide.ad.jp/mawi/

40 usenix.org
https://www.usenix.org/system/files/sec21-liu-zaoxing.pdf

45 Network-Wide Heavy Hitter Detection with Commodity Switches
https://www.cs.princeton.edu/~jrex/papers/net-heavy18.pdf